

OTA V4. 1. X 使用和协议说明

(软件版本 V4. 1. 12)

(适配工具版本号 V1. 3. 6)

文件修改历史			
文档版本号	发起者	描述	日期

V1.0	启英泰伦	新建文档	2026/07/06
V1.1	启英泰伦	1. 支持升级固件只打包部分分区	2026/07/09

目录

OTA V4.1.X 使用和协议说明	1
1. OTA 方案介绍	4
1.1 概述	4
1.2 适用对象	4
2. 术语和定义	4
3. 功能特性说明	4
4. 系统通信及协议设计	6
4.1 交互流程	6
4.2 SDK 配置及说明	8
4.3 通信协议	9
5. windows 工具使用说明及使用方法	12
5.1 OTA bin 文件生成	12
5.2 OTA Tool V1.3.X 工具说明	13
6. 主端参考 demo 说明	19
enter_ota_mode()	19
ota_check_version()	19
ota_start()	19
ota_transport_firmware()	19
ota_end()	20
7. OTA 适配重点	20
7.1 流程如何设计?	20
7.2 设备未进入 OTA 模式	20
7.3 OTA 升级失败	21
7.4 低波特率 OTA	21
7.5 OTA 阶段 log 和升级串口使用同一个	21
7.6 使用自定义的串口引脚	21
7.7 修改超时时间和次数	22
7.8 主端参数说明	22
8. 旧版 sdk 或用户自定义适配流程	23

8.1 语音端进入 ota	23
---------------------	----

1. OTA 方案介绍

1.1 概述

本文档适用于通过串口对启英泰伦语音芯片 sdk 固件进行 OTA 升级的方案；升级方式为串口升级，默认升级串口号、波特率和语音通信协议一致。

1.2 适用对象

启英泰伦 FAE 团队、语音方案二次开发人员。

2. 术语和定义

术语	解释
OTA	Over-The-Air，空中升级，本文特指通过串口进行固件更新
主程序	由正常功能的用户使用程序
OTA 程序	执行 OTA 功能的程序
主端	OTA 流程中主控端，负责下发固件，控制 OTA 流程，如主端、PC 工具
语音端	OTA 流程中从端，负责接收固件并写入 Flash，本文指我司语音芯片模块
dev	下挂在语音端串口上的电控或其他 mcu

3. 功能特性说明

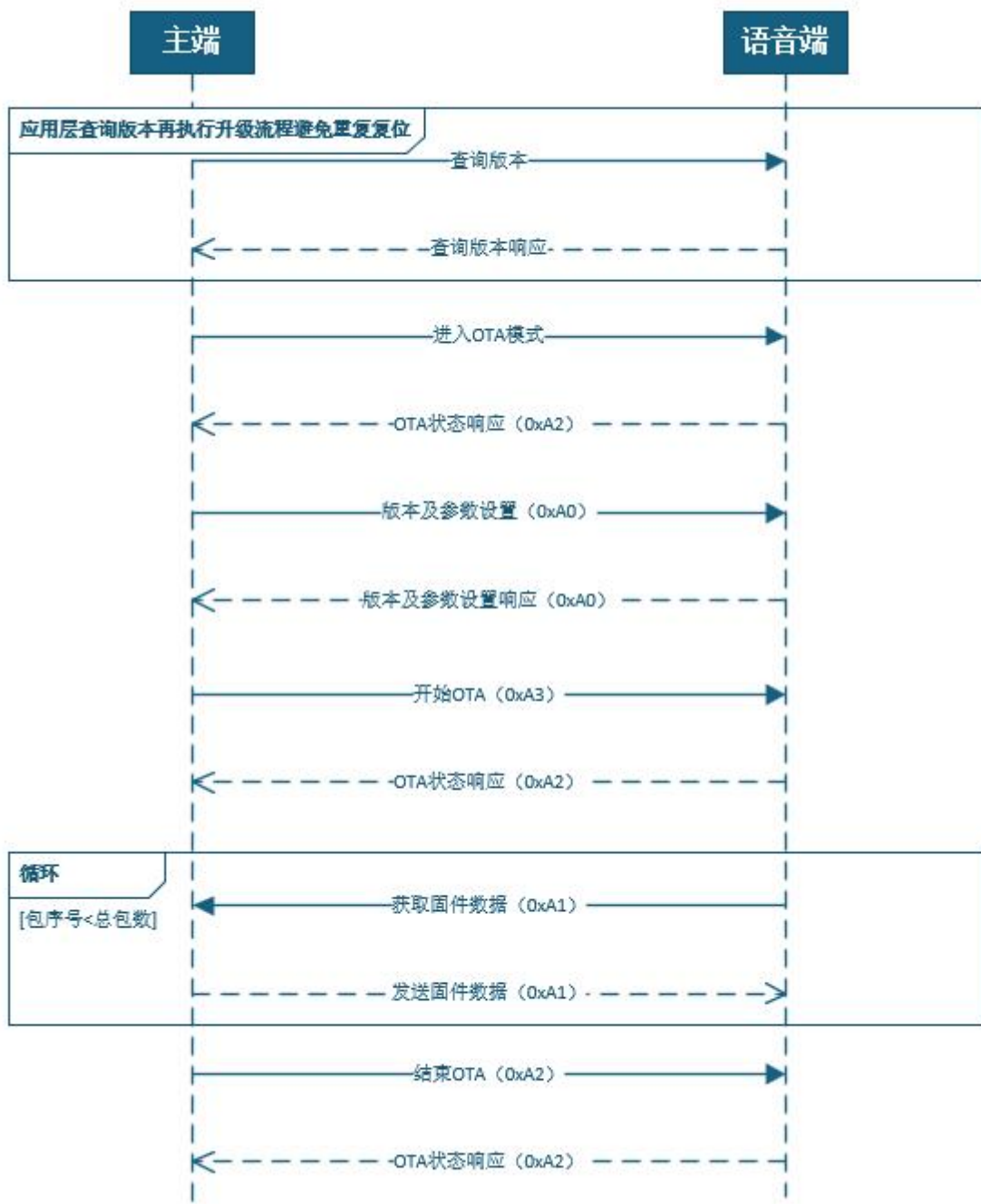
- 支持全量固件升级：支持全量升级 (usercode2、asr、dnn、voice、userfile)。
- 支持部分分区升级：支持升级固件只打包部分分区。
- 支持跳区升级：升级时自动校验差异分区，只下载有差异的分区。
- 支持分区级续传：升级中断时，复位后再次触发升级从上次的分区开始。
- 运行自恢复：进入 OTA 模式超过一定时间没有收到主端 OTA 命令，若主程序有效，清除 OTA 标志，跳转执行主程序。
- 固件校验：支持主端定义校验内容，非法固件禁止升级。

➤ 产品编号

- 芯片型号
- 软件版本号
- 硬件版本号
- CRC 错误固件
- 固件大小
- 分区版本号
- 断电保护：升级过程中若发生断电或通信中断，重新上电后能自动识别固件无效，重新进入 OTA 模式，避免设备变砖。
- 超时重传：升级过程中命令、数据在超时时间内（可设置，参考 [7.7 修改超时时间和次数](#)）未收到响应或者下一包数据，进行重传。
- 有限重试：超时重传只执行三次（可设置，参考 [7.7 修改超时时间和次数](#)），三次内不能成功则认为 OTA 失败，语音端和主端重新执行 OTA 流程。
- OTA 模式通告：若设备异常进入 OTA 模式，将会重复发送通告消息（进入 OTA 模式响应）给主端。

4. 系统通信及协议设计

4.1 交互流程



备注:在 OTA 之前,主端可以先查询语音端版本号,版本号符合升级需求再下发进入 OTA 模式命令

交互说明:

1. 主端发送协议,语音端进入 OTA 模式

2. 语音端收到协议后，写 OTA 的标记信息到 NV，然后复位芯片进入 OTA 模式，发送语音端 OTA 状态给主端。
3. 主端发送 OTA 版本及参数设置（0xA0），同时设置语音端是否检查产品型号、芯片型号、版本号、NV 地址、分区版本号、波特率（0 表示不设置波特率）等参数。
4. 语音端回复 ACK 是否允许 OTA。
5. 主端收到允许 OTA 则下发开始 OTA 消息，否则下发结束 OTA 消息。
6. 语音端开始 OTA 后，设置波特率（如果要设置），回复语音端 OTA 状态，然后发送获取 OTA 数据的消息。
7. 主端收到后，按照设置的帧长度，回复语音端 OTA 数据。
8. 语音端收到 OTA 数据，做数据校验，并写入到 flash。
9. 主端收到请求包大于固件总包数，发送结束 OTA 消息。
10. 语音端校验分区表和分区信息，判断 OTA 是否成功。若成功，擦除 OTA 使能标记，回复 OTA 成功，否则回复 OTA 失败，然后复位系统。

OTA 特性：

1. 进入 OTA 模式后，不支持语音识别
2. OTA 时，语音端异常断电，需要主端重新发起 OTA
3. 开始进入 OTA，并且分区信息发送和校验成功后，OTA 异常结束，程序停留在 OTA 模式，必须 OTA 成功后，才能切换回正常模式，进行语音识别
4. 语音端在升级之前，做了如下的逻辑检查：
 - (1) updater 分区校验：若 updater 程序大小超过 updater 分区大小，语音端回复错误，拒绝 OTA。
 - (2) 固件软件版本号校验：主端开启了固件软件版本号校验，若版本号一致，语音端回复错误，拒绝 OTA。
 - (3) 固件硬件版本号校验：主端开启了固件硬件版本号校验，若版本号不一致，语音端回复错误，拒绝 OTA。
 - (4) FLASH 大小校验：主端开启了 FLASH 大小校验。若 FLASH 大小错误，语音端回复错误，拒绝 OTA。
 - (5) 产品号校验：主端开启了产品号检查，若产品号不符，语音端回复错误，拒绝 OTA。

(6) 芯片型号校验：主端开启了芯片型号检查，若芯片型号不符，语音端回复错误，拒绝 OTA。

(7) 固件合法性校验：打包新的固件时，分区信息有改变；制作 OTA 固件时，未选择有改变的分区。语音端回复错误，拒绝 OTA。

上述情况，OTA 异常，主端收到错误后，需发送 OTA 结束指令，语音端收到后复位，版本信息不更新。

语音端开始擦除分区，并写入新的 OTA 固件信息后，会做如下检查：

(1) 收到消息帧 crc 校验错误，或者长时间未收到主端发送的 OTA 固件消息，语音端重复发送获取 OTA 数据协议 (0xA1)。若重试次数超过设置次数，语音端复位后重新进行 OTA 模式。

(2) 语音端写入每个分区完成后，会做分区完整信息的 crc16 校验，若校验不通过，语音端回复错误。主端需要检查 OTA 固件信息是否正确，重新发起 OTA。

上述情况，OTA 异常，主端收到错误后，需检查 OTA 固件信息或者传输过程是否有误，重新发起 OTA。语音端进入 OTA 模式后，升级成功前一直处于 OTA 模式。

4.2 SDK 配置及说明

主端可以配置 OTA 的波特率和校验内容，参考主端工程配置文件，使用 OTA 功能需要在语音芯片端 SDK `user_config.h` 文件中使能该配置；OTA 通信串口、主端通信串口使用同一个串口和串口波特率；主端在版本及参数设置命令中可以更改串口及波特率。宏定义如下：（这段话再改下）

功能	宏定义	备注
OTA 功能使能	CI_OTA_ENABLE	语音端 sdk 配置
通信串口	UART_PROYTOCOL_NUMBER	通信串口和主端串口保持一致
波特率设置	UART_PROTOCOL_BAUDRATE	通信和主端波特率保持一致
OTA 串口波特率	OTA_UART_BAUDRATE	921600
每包数据长度	OTA_PACK_LENGTH	4096
固件软件版本号检查	OTA_CHECK_SOFT_VER	1
固件硬件版本号检查	OTA_CHECK_HARD_VER	1
硬件 Flash 大小检查	OTA_CHECK_FLASH_SIZE	1
产品编号检查	OTA_CHECK_PID	1
芯片类型检查	OTA_CHECK_CHIP_TYPE	1
固件分区版本号检查	OTA_CHECK_PART_VER	0
OTA 固件传输波特率	OTA_UART_UPDATE_BAUDRATE	0-2000000（设置 0 使用

		默认波特率)
--	--	--------

4.3 通信协议

进入 OTA 模式命令，不同的 sdk 指令不同，支持用户自定义，公版示例指令：

主端发送到语音端进入 OTA 模式命令	
算法 SDK	0xA5 0xFC 0x00 0x00 0xA1 0x97 0x00 0x38 0x01 0xFB
AIOT SDK	0xA5 0xA5 0x5A 0x5A 0x00 0x00 0x01 0x05 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00

注：其中 0x97 和 0x0501 是 sdk 中进入 OTA 模式的判断字段。

OTA 通信协议如下，分为主端(pc 或者 app)发送的协议和语音芯片发送的协议：

注意：

- ① 帧长为帧类型+帧数据+CRC+结束符的长度。
- ② CRC 校验为帧头+帧长+帧类型+帧数据的 CRC16。
- ③ 中继下发和中继上传的数据需要与协议帧格式一致，crc 校验失败会丢弃。

主端发送到语音芯片						
	帧头 (2 字节)	帧长 (2 字节)	帧类型 (1 字节)	帧数据	crc16 校验 (2 字节)	结束符 (1 字节)
版本及参数设置命令	0xA50F	帧类型长度+数据长度+crc 长度+结束符	0xA0	是否校验软件版本号 (1 字节) +是否校验硬件版本号 (1 字节) +是否校验产品型号 (1 字节) +是否校验芯片型号 (1 字节) +是否校验 nv 地址 (1 字节) +是否校验分区版本号 (1 字节) +OTA 的固件版本号 (3 字节) + 波特率 (4 字节) 是否校验： 不校验：0x00 校验：0x01 固件版本号： 高字节在前序， 例如 V1.2.3 版本，数据传输为 0x01 0x02 0x03 波特率： 高字节在前 超时时间&&超时次数： 不设置：0x00 其他值：高字节在前	帧头+数据长度+帧类型+帧数据的 CRC16： XXXX	0xFF
发送 OTA	0xA50F	帧类型长度+数据长度+crc	0xA1	包序号 (2 字节) + 每包有效数据	XXXX	0xFF

升级固件数据		长度+结束符				
发送 OTA 结束指令	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA2	0 字节	XXXX	0xFF
发送 OTA 开始指令	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA3	OTA 语音端: 0x00 OTA 下挂设备: 0x01	XXXX	0xFF
中继参数设置	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA4	中继参数设置 (串口号、波特率)	XXXX	0xFF
				该组参数用于设置语音端与下挂设备通信串口 串口号: 1 字节 波特率: 4 字节, 高字节在前		
中继下发	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA5	中继数据	XXXX	0xFF
语音芯片发送到主端						
	帧头 (2 字节)	数据长度 (2 字节)	帧类型 (1 字节)	帧数据	crc16 校验 (2 字节)	结束符 (1 字节)
版本及参数设置响应	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA0	校验结果+ 版本信息 (3 字节) +拒绝原因 (1 字节)	XXXX	0xFF
				校验结果: 拒绝 OTA: 0x00 允许 OTA: 0x01 失败原因: 软件版本号错误: 0x01 硬件版本号错误: 0x02		

应				波特率错误: 0x03 NV 地址错误: 0x04 产品型号错误: 0x05 芯片型号: 0x06 分区 CRC 错误: 0x07 分区表 CRC 错误: 0x08 固件头信息错误: 0x09 固件大小错误: 0x0A ota code 大小错误: 0x0B 变化分区未升级: 0x0C 分区版本号错误: 0x0D 消息超时: 0x0E 保留: 0x0F		
固件请求	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA1	0x00: 写入失败(1 字节) + 失败原因 (1 字节); 0x01: 写入成功(1 字节) + 当前包序号 (2 字节) + 下一包包序号 (2 字节)	XXXX	0xFF
OTA 状态响应	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA2	0x00: 升级失败; 0x01: 升级成功; 0x02: 等待升级; 0x03: 升级语音中; 0x04: 升级下挂设备中+原因 (1 字节) 当升级失败和等待升级时需要传入原因。 升级失败原因: 参考版本及参数设置 等待升级原因: 主端主动发起: 0x01 语音端分区错误: 0x02 加载用户 code 失败: 0x03 保留: 0x04	XXXX	0xFF
中继参数设置响应	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA4	拒绝 OTA: 0x00 允许 OTA (串口或者波特率错误): 0x01	XXXX	0xFF
中继上报	0xA50F	帧类型长度+数据长度+crc长度+结束符	0xA6	中继数据 最大长度 4KB+10	XXXX	0xFF

5. windows 工具使用说明及使用方法

5.1 OTA bin 文件生成

5.1.1 生成 updater 原始 bin 文件

5.1.2 通过源码生成文件

通过 ota_code 源码生成 ota_code.bin; 这里可能涉及到芯片型号、日志串口和 OTA 串口; 编译方法与其他 sdk 一致; 通过修改 user_config.h 文件配置参数, 工具中执行编译;

芯片型号:

```
user_config.h
projects > offline_asr_sample > src > @ user_config.h > ...

1  #ifndef __USER_CONFIG_H__
2  #define __USER_CONFIG_H__
3
4
5  /**板级配置选择
6  /*板级配置更多细节请查看:https://document.chipintelli.com/硬件资料-->模块手册
7  chipintelli提供的部分开发板和模组, 可以通过下面的宏选择, 也可以参考开发板的板级配置文
8  件添加自定义板级配置文件*/
9
10 //OTA V4.1不需要修改板级, 默认使用1306配置, 若串口引脚不一致, 在对应配置文件中适配, OTA启动时读取芯片类型自动适配.
11 #define USE_CUS_OTA_BOARD_1302 0 //CI-D06GT010, 开发版, 适配芯片1301、1302、1303、1312
12 #define USE_CUS_OTA_BOARD_1306 1 //CI-D06GT010, 开发版, 适配芯片1306等
13
14 #if (USE_CUS_OTA_BOARD_1302 == 1)
```

串口:

```
CI TOOL
offline_asr_sample
清理
生成
设为活动工程
设置烧录端口号
生成分区bin文件
自动打包固件工具
烧录usercode Ctrl+Rn

user_config.h
projects > offline_asr_sample > src > @ user_config.h > ...

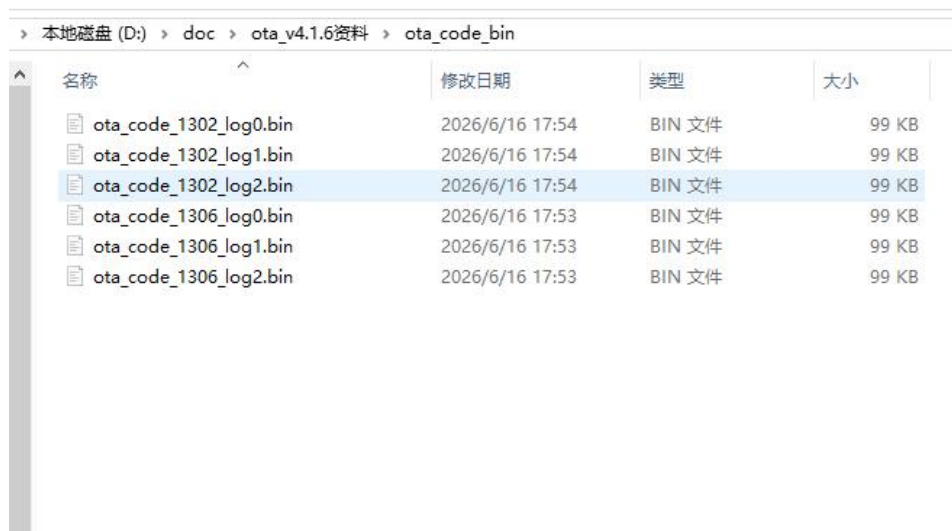
39 #ifndef __USER_CONFIG_H__
40 #define __USER_CONFIG_H__
41 #endif
42
43 #define OTA_PARTITION_CHECK_EN 0 //1, 开启分区严格检查, 0, 关闭分区严格检查.
44
45 /**麦克风电路模式配置
46 #define MIC_DIFF_SINGLE 0 //+1, 单端, 0, 差分 (通用模块都是差分模式, 宝德本的模块为单端
47 (MICN_L 接GND) 时, 需要配置为SINGLE) */
48
49 /**IIS1515音频功能开关配置
50 #define USE_IIS1515_OUT_PSE_RSLT_AUDIO 0 //1, 开启IIS1515音频功能, 可以使用语音识别, 占用PA2~PA6.
51 //0, 关闭IIS1515音频功能, PA2~PA6可以用于其它功能.
52
53 /**通信串口配置
54 #define CONFIG_CI_LOG_UART HAL_UART0_BASE //配置log输出使用的串口
55 #define MSG_COM_USE_UART_EN 1 //0, 关闭语音模块通讯协议, 1, 开启语音模块通讯协议.
56 #define UART_PROTOCOL_NUMBER (HAL_UART3_BASE) //语音模块协议使用的串口.
57 #define UART_PROTOCOL_BAUDRATE (UART_BaudRate921600) //语音模块协议使用的串口波特率.
58 #define UART_PROTOCOL_VER 2 //语音模块协议版本号: 1, 一代协议; 2, 二代协议; 255, 平台生成协议
59
60 #define OTA_TIMEOUT 3 //数据超时时间, 单位秒
61 #define OTA_RETRY_TIME 3 //数据重发次数
62
63 #if (USE_CI_D06GT010_BOARD == 1)
64 #define OTA_CHIP_TYPE_1306 1
65 #else
66 #define OTA_CHIP_TYPE_1306 0
67 #endif
68
69 /**通信接口引脚配置
70 //注: 推挽模式的IO只能对接3.3V电平的IO, 开漏模式可以对接5V电平的IO (外部需要上拉到5V)
71 #define UART0_PAD_OPENDRAIN_MODE_EN 0 //0, UART0为推挽模式, 1, UART0为开漏模式.
72 #define UART1_PAD_OPENDRAIN_MODE_EN 0 //0, UART1为推挽模式, 1, UART1为开漏模式.
73 #define UART2_PAD_OPENDRAIN_MODE_EN 0 //0, UART2为推挽模式, 1, UART2为开漏模式.
74
75 //时钟配置
76 #if (CI_CHIP_TYPE == 1312) || (CI_CHIP_TYPE == 1311)
77 #define USE_EXTERNAL_CRYSTAL_OSC 0
```

编译完成后会在 projects\offline_asr_sample\firmware\user_code 生成

ota_code_13xx_log0.bin, 这个 bin 文件就是 updater 参数配置中需要的源固件;

5.1.3 使用资料包中自带配置

资料包中已经有了基于源码生成的 ota_code.bin, 如果没有特殊需求可以使用资料包中自带的, 其中 log0 代表日志使用串口 0。如下:



名称	修改日期	类型	大小
ota_code_1302_log0.bin	2026/6/16 17:54	BIN 文件	99 KB
ota_code_1302_log1.bin	2026/6/16 17:54	BIN 文件	99 KB
ota_code_1302_log2.bin	2026/6/16 17:54	BIN 文件	99 KB
ota_code_1306_log0.bin	2026/6/16 17:53	BIN 文件	99 KB
ota_code_1306_log1.bin	2026/6/16 17:53	BIN 文件	99 KB
ota_code_1306_log2.bin	2026/6/16 17:53	BIN 文件	99 KB

5.2 OTA Tool V1.3.X 工具说明

5.2.1 updater 参数配置

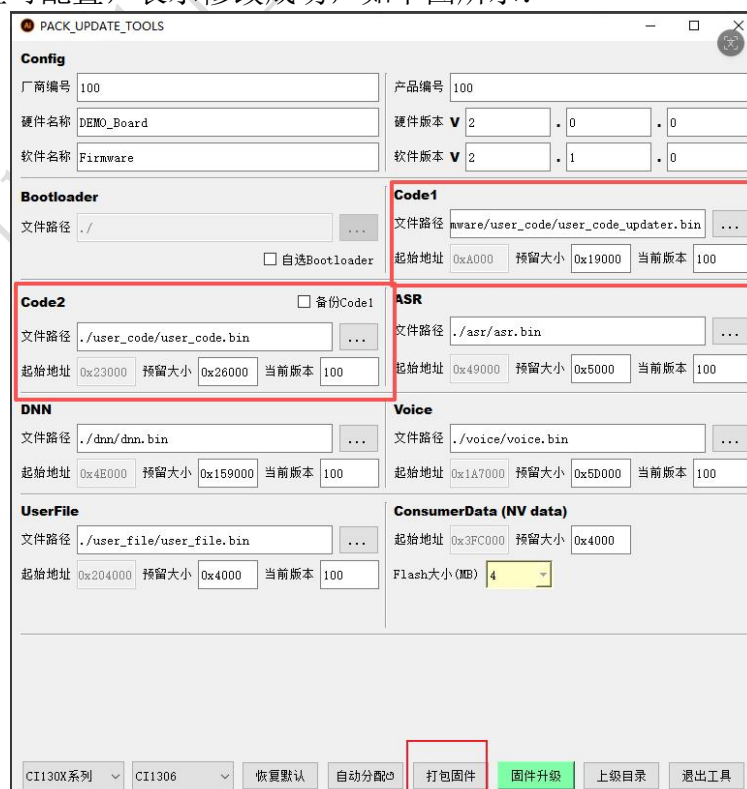
配置 ota 过程中的串口号、波特率、芯片型号、用户协议的帧结构、用户协议的帧类型等参数, 方便客户使用自己的协议来完成 ota 适配。

- 源固件, ota code 源码生成的 ota 程序
- 目标固件, 使用 ota code 生成的带配置的 updater 程序
- ota 参数设置, 设置 ota 过程中的串口号、波特率、芯片型号
- 帧结构设置, 勾选此选项可以开启用户协议与标准协议的转换, 大小端影响帧长、crc 的读取, 帧长最大支持 4 字节, crc 只支持标准协议中的 crc16;
- 配置帧类型, 勾选配置帧类型可以开启用户协议帧类型与标准协议帧类型的映射, 若帧类型未修改不用勾选。



5.2.2 PACK_UPDATE_TOOLS 工具生成固件包（使用 4.0.5 1.0 及以上版本）

固件打包需修改 firmware 文件夹下面 config.ini 文件:firmware_version=FW_V4。修改好后,重新打开 PACK_UPDATE_TOOLS,工具里面合成固件的 code1 (OTA_code) 和 code2 (user_code) 地址可配置,表示修改成功,如下图所示:

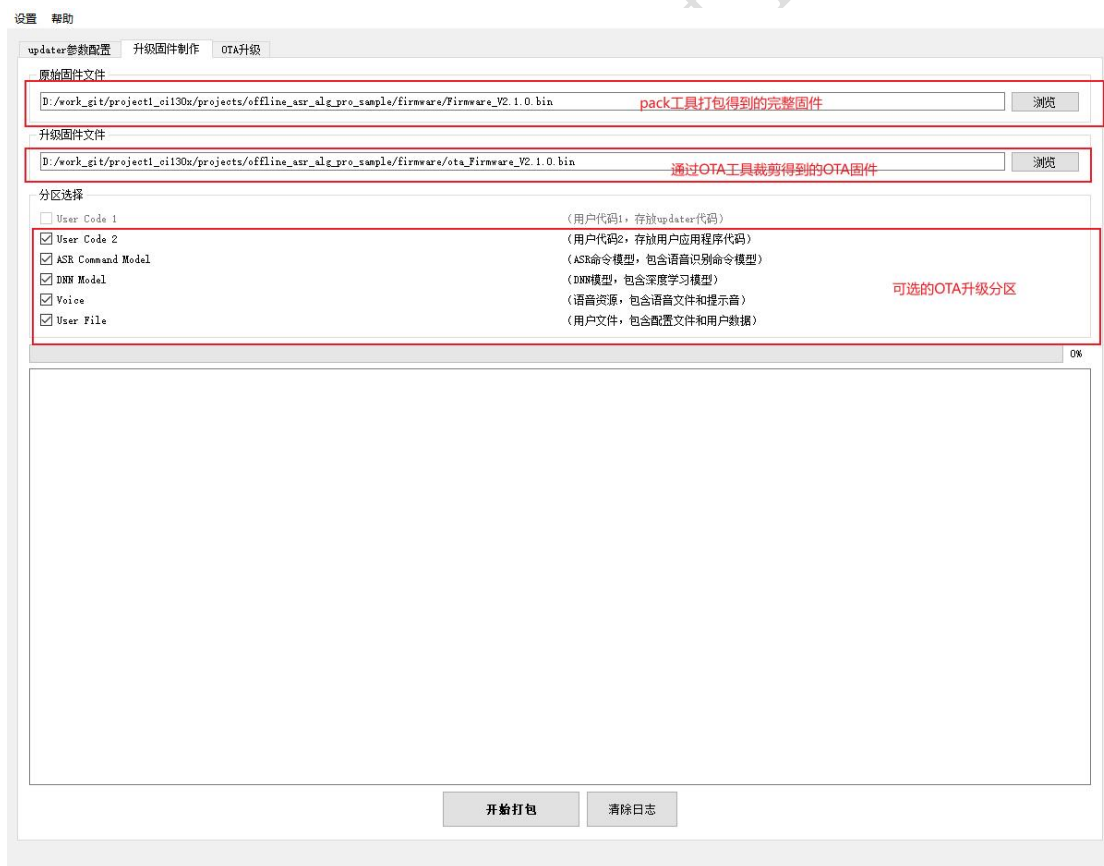


注：打包时请勿勾选备份 code1

打包固件包含 boot、分区表、备份分区表、ota 程序、主程序等，OTA 程序暂时不可升级；内容如下表：

分区起始地址	分区
0x000	Boot
0x6000	分区表
0x8000	备份分区表
0xA000	ota 程序固件
0xA000+OTA 程序固件大小	use_code2 分区
user_code2 起始地址+user_code2 大小	asr 分区
asr 起始地址+asr 大小	dnn 分区
dnn 起始地址+dnn 大小	voice 分区
voice 起始地址+voice 大小	userfile 分区
0xFC000/3FC000	NV 分区 (2M/4M FLASH)

5.2.3 OTA 升级固件制作



OTA 固件头信息用于记录需要升级的内容，格式如下。

- 固件头（4KB，大端格式）：
长度（2 字节，有效数据长度不含 CRC）

分区数（2 字节，高字节在前）

可选升级分区标志（字符串表示，例如 usercode、asr...）

CRC16（校验分区数和分区标志）

- 分区表（从 0x1000 开始）
- 可选分区内容（从 0x2000 开始，依次为 user_code2、asr、dnm、voice、userfile）

5.2.3 OTA 升级模拟

通过工具模拟 OTA 升级，支持设置串口、传输波特率、等参数，操作步骤如下：

- (1) 设置串口和波特率，连接串口。
- (2) 设置 ota 流程中的参数。
- (3) 设置进入 ota 模式的方法。
- (4) 选择升级文件。
- (5) 点击开始 ota，等待运行。
- (6) 运行中会打印各种流程信息。
- (7) 工具目录下会自动保存对应时间戳的升级 log。

设置 帮助

updater参数配置 升级固件制作 OTA升级

串口设置

串口: COM9 默认波特率: 921600 连接 语音板连接的端口和初始波特率

参数设置

传输波特率: 2000000 单包最大长度: 512B

☐ 检查软件版本 ☒ 检查硬件版本 ☒ 检查产品编号 ☐ 检查分区版本

☒ 检查芯片类型 ☒ 检查flash大小

OTA过程中固件传输时的波特率、单包大小、校验参数

进入OTA模式方式

☒ 算法SDK (cmd: 0x97) ☐ 离在线SDK (cmd: 0x0501) 哪个sdk

自动执行OTA

☐ 启用自动执行 执行间隔(秒): 30 自动执行可以测试稳定性

语音升级

升级文件1: D:/work_git/project1_ci130a/projects/offline_asr_alg_pro_sample/firmware/ota_firmware_V2.1.0.bin 浏览

升级文件2: 升级文件，如果开启自动执行可以在循环升级两个文件，如果只有一个文件，会重复升级

协议配置

配置文件: 未选择配置文件时使用标准协议 浏览 清除 协议配置文件，如果用户协议与标准协议不一致，这里需要导入updater参数设置中生成的配置文件

0% 00:00

开始OTA 清除日志

5.2.4 用户协议转换

当用户使用固定帧格式的消息且校验方式与我司一致时，可以使用协议转换功能，快速完成协议适配；

备注：用户协议中帧类型与标准协议要有相对应的，中继不使用时可以没有；

界面如下：

用户协议如下：

帧头	版本	帧序号	帧类型	帧长	帧数据	crc
A5 0F	02	1 字节	2 字节	2 (n 字节+crc 长度)	n 字节	2 字节

帧类型定义：

备注：上报帧与上报帧、下发帧与下发帧不允许相同，中继相关如果没用到可以设置为不使用的值；

数据长度补充是协议字段中帧长与数据长度之间的差值，当前例子中帧长是数据长度 n 字节+crc2 字节，所以数据长度补充就是 2；

6. 主端参考 demo 说明

主端 demo 可以参考 [OTA V4.1 主端参考代码 linux 工程](#);

`enter_ota_mode()`

主端发送进入 ota 模式命令到语音端，发送后等待语音端回复 OTA 状态响应（等待升级状态）。收到响应说明语音端进入 OTA 模式。具体命令参考[通信协议章节](#)。

`ota_check_version()`

主端发送 ota 版本及参数设置命令到语音端，语音端保存设置参数，并校验版本是否正确。返回校验结果给主端。主端根据返回结果决定 OTA 后续流程。

`ota_start()`

主端发送开始 ota 命令给语音端，语音端收到命令后，更新通信波特率为版本及参数设置命令中的值（波特率为 0 时，为默认值），然后回复 OTA 状态响应（升级语音端状态或升级下挂设备状态）

`ota_transport_firmware()`

发送 OTA 升级固件有效数据命令，包含两段参数：

- 1: 包序号，第一次发送时为 0，后续的包序号，从语音端回复的 ACK 里面解析得出。
- 2: 有效数据，从待升级固件 OTA_INVALID_HEAD_LENGTH 开始，加上包序号乘以每包有效数据长度（OTA_PACK_LENGTH），开始读固件并取出。

主端发送数据到收到语音端的超时时间需要设置高一点，至少大于 5S（语音端擦除 flash 比较耗时）。默认的策略是发送第一次超时未收到 ACK，重发一次，重发也未收到 ACK，程序判断 OTA 失败。

语音端返回的固件传输 ACK 消息有两个状态。

1: 如果状态是 0，表示写入失败。后续第一个字节表示写入固件失败的原因，有如下值：-1，当前系统内存不足导致不能继续 OTA；-2，下发固件不是 OTA 固件；-3，下发的固件分区表信息校验不通过；-4，下发的固件硬件版本信息不一致；-5，下发固件后校验分区的 crc 不正确。

2: 如果返回状态是 1，表示写入成功，解析返回数据的下一包包序号字段，确认发送的下一包数据内容。特别注意，如果返回的下一包包序号大于了主端计算的数据包总

包数，表示固件全部传输完成，主端程序判断 OTA 成功，发送 OTA 结束指令。

ota_end()

发送 OTA 结束指令，语音端收到后会回复 OTA 状态响应（OTA 成功或失败）并立即软件复位重启。主端结束 OTA 逻辑回到语音协议交互逻辑。

7. OTA 适配重点

7.1 流程如何设计？

1. 主端通过语音协议先获取语音端软件版本号，校验版本号是否需要升级；满足升级需求再下发进入 OTA 命令；这样可以避免语音端重复进入 OTA 模式。
2. 语音端 OTA 成功后会将其中标志设为 OTA 成功，如果有要求 OTA 成功后正常进入语音识别模式进行播报或者上报消息，可以通过读取标志位来实现。
3. 语音端默认接收消息和发送都会有 3s 超时，重试 3 次；主端在发送消息时可以参考 demo 实现超时（4s 和 3 次）和重试；
4. 语音端在收到切换波特率命令时，会在 50ms 之后发出数据，主端可以在数据发送后延迟 10ms 等待发送完成再切换波特率等地啊接收语音端消息；
5. 若用户已有协议，通过工具实现协议转换通信。标准 OTA 流程中的协议若用户没有需要新增。
6. 使用 ota_code 源码编译出 user_code 文件；可以直接使用 ota_code_bin 目录下的 bin 文件，选择串口对对应的版本即可。
7. 使用打包工具打包程序，打包工具配置文件要设为 FW4，，user_code1 选择步骤 4 生成的文件。

路径：projects\offline_asr_alg_pro_sample\firmware\config.ini

7.2 设备未进入 OTA 模式

进入 OTA 模式需要通过 flash 中的标记来判断，应用层需要在消息处理时写入 flash 标记；

通过向语音端发送串口协议（可以自定义），接收到消息时调用 write_ota_mcu_status 函数即可使设备进入 OTA 模式。

调用时：

```
// OTA_STA_ENTER 进入 ota 模式
```

```
write_ota_mcu_status(OTA_STA_ENTER);
```

语音执行此函数进入使设备下次重启后，进入 OTA 模式，其中 NVDATA_ID_OTA_MCU_STATUS 为 NV 区 OTA 标记的地址，默认 0x80000001。不可修改；OTA 标志枚举如下：

```
typedef enum
{
    OTA_STA_FAIL = 0x00,      // OTA 失败
    OTA_STA_SUC = 0x01,      // OTA 成功
    OTA_STA_WAITING = 0x02,  // 等待 OTA
    OTA_STA_CI = 0x03,       // OTA 语音端
    OTA_STA_DEV = 0x04,      // OTA 下挂设备中
    OTA_STA_ENTER = 0x05,    // 进入 OTA
} ota_state_t;
```

函数实现如下：

```
cinv_item_ret_t write_ota_mcu_status(uint8_t status)
{
    uint8_t flag = status;
    return cinv_item_write(NVDATA_ID_OTA_MCU_STATUS, sizeof(uint8_t), &flag);
}
```

7.3 OTA 升级失败

OTA 升级流程中发生错误，语音设备都有详细的错误类型上报，可以查看[通信协议](#)中的失败原因列表。

7.4 低波特率 OTA

使用较低的波特率 OTA 时，需要调整传输的超时时间和单包大小，例如 9600 波特率时，若使用 4k 单包来传输，必然发生超时，理论速率传输完 4k 报文，需要 5s 多时间；不建议使用较低波特率，升级时间较长。

7.5 OTA 阶段 log 和升级串口使用同一个

不建议使用同一个，如果一定要使用同一个，需要保证 log 打印和消息发送时的时序；多任务运行时可能冲突导致消息发送失败；

7.6 使用自定义的串口引脚

如果需要使用自定义串口引脚，需要在 ota_code 源码中修改板级配置文件；语音 code 和 ota code 同步修改；ota code 固定加载 driver\boards\CI-D06GT01D.c

修改其中函数 void pad_config_for_uart (UART_TypeDef *UARTx) 中对应串口的引脚即可。

7.7 修改超时时间和次数

由于主端可能通过 wifi 或 ble 来执行升级流程，交互需要经过云端、主端、语音端，此时 3s 的超时时间就不太够用，可以通过修改源码中的配置来设置更长的超时时间。

修改 user_config 中的 OTA_TIMEOUT 和 OTA_RETRY_TIME：

```
user_config.h
17 #define CI_CHIP_TYPE 1386 //Flash:4MB, QFN40
18 #define BOARD_PORT_FILE "CI-086GT010.c"
19 #define CI_XXXXXX_BOARD 1
20 #define CI_CHIP_TYPE xxxx //Flash:4MB, QFN40
21 #define BOARD_PORT_FILE "CI-XXXX.c"
22 #endif
23
24 #define OTA_PARTITION_CHECK_EN 0 //1, 开启分区严格检查。0, 关闭分区严格检查。
25
26 /**+麦克风电路模式配置
27 #define MIC_DIFF_SINGLE 0 /*1, 单端。0, 差分（通用模块都是差分模式，省成本的模块为单端
28 (MICN_L 接GND) 时，需要配置为SINGLE）*/
29
30 /**+IIS语音功能开关配置
31 #define USE_IIS_OUT_PRE_RSLT_AUDIO 0 //1, 开启IIS语音功能，可以使用音频板录音，占用PA2~PA6。
32 //0, 关闭IIS语音功能，PA2~PA6可以用于其它功能。
33
34 /**+语音模块配置
35 #define CONFIG_CI_LOG_UART HAL_UART0_BASE //配置Log输出使用的串口
36 #define MSG_COM_USE_UART_EN 1 //0, 关闭语音模块通讯协议。1, 开启语音模块通讯协议。
37 #define UART_PROTOCOL_NUMBER (HAL_UART1_BASE) //语音模块协议使用的串口。
38 #define UART_PROTOCOL_BAUDRATE (UART_BaudRate921600) //语音模块协议使用的串口波特率。
39 #define UART_PROTOCOL_VER 2 //语音模块协议版本号:1, 一代协议。2, 二代协议。255, 平台生成协议
40
41 #define OTA_TIMEOUT 3 //数据超时时间，单位秒
42 #define OTA_RETRY_TIME 3 //数据重复发送次数
43 #if (USE_CI_086GT010_BOARD == 1)
44 #define OTA_CHIP_TYPE_1386 1
45 #else
46 #define OTA_CHIP_TYPE_1386 0
47 #endif
48
```

7.8 主端参数说明

可选配置参数如下图：

```
C:\ias_ota_config\2
C:\ias_ota_config\2
1 #define __CIAS_OTA_CONFIG_H__
2 #define __CIAS_OTA_CONFIG_H__
3 #include <stdint.h>
4 #include <stdbool.h>
5 #define DEBUG_LOG 1
6 #define OUT_DEBUG_INFO 1
7
8 #define OTA_INVALID_HEAD_LENGTH 0x1000 // ota固件的分区表信息位置
9 #define OTA_UART_BAUDRATE 921600 // 默认波特率
10 #define OTA_PACK_LENGTH 4096 // 每包有效数据长度，只支持4KB的约数，例如1KB, 2KB, 4KB
11 #define OTA_UPDATE_BAUDRATE 1000000 // 固件传输波特率，最大2M，为0时不更新
12 #define OTA_UART_DEV_NUM 0 // 下挂设备串口号0~2
13 #define OTA_UART_DEV_BAUDRATE 1000000 // 下挂设备波特率
14
15 #define OTA_CHECK_SOFT_VER 1 // 语音端是否检查ota软件版本号，0为不检查，强制ota；1为需要检查，版本不一致才能ota
16 #define OTA_CHECK_HARD_VER 1 // 语音端是否检查ota硬件版本号，0为不检查，强制ota；1为需要检查，版本一致才能ota
17 #define OTA_CHECK_PID 1 // 语音端是否检查ota产品编号，0为不检查，强制ota；1为需要检查，产品编号一致才能ota
18 #define OTA_CHECK_CHIP_TYPE 1 // 语音端是否检查ota芯片类型，0为不检查，强制ota；1为需要检查，芯片类型一致才能ota
19 #define OTA_CHECK_FLASH_SIZE 1 // 语音端是否检查ota固件flash大小是否一致，0为不检查，强制ota；1为需要检查，flash大小一致才能ota
20 #define OTA_CHECK_PART_VER 0 // 语音端是否检查ota固件分区版本号，0为不检查，强制ota；1为需要检查，分区版本号不一致才能ota
21
```

7.9 升级固件制作说明

升级固件建议选择所有分区，升级时会自动判断哪些分区需要升级，升级时间与部分分区升级差异不大；如果需要升级固件比较小可以选择打包部分分区。

8. OTA V4.1.5 如何更新到 V4.1.12

OTA V4.1.12 兼容 OTA V4.1.5 的升级，同时解决了许多升级时的 bug；所以建议用户升级 V4.1.5 到 V4.1.12，更新时参考如下流程：

1. 更新当前 sdk 中的 updater 程序，使用 V4.1.12 中的 ota code 源码编译的 bin 替换旧的 bin 文件；



2. 更新主端程序

主端参考 demo 实现，如果是 linux 系统可以直接使用 demo；其他系统参考 demo 实现；

注意：需要在固件发送阶段需要按照报文请求中的包序号去获取对应的固件内容；

例如第 200 包，则对应固件中 $200 \times \text{每包大小的偏移地址}$ ；

9. 旧版 sdk 或用户自定义适配流程

9.1 语音端进入 ota

语音端发送协议命令进入
在语音端执行



语音端公版进入到 ota 模式命令：A5 FC 00 00 A1 97 00 38 01 FB

```
mprintf("Enter ota mode\r\n");
vTaskDelay(pdMS_TO_TICKS(15)); // 加点延迟 否则打印不完整
write_ota_mcu_status(OTA_STA_UPDATE);
dpmu_software_reset_system_config();
```

注意事项:

1. OTA_STA_UPDATE 这个是语音端进入 ota 前保存，他的作用：是否在 ota-code 进入 ota 升级还是非升级状态；客户使用公版的 sdk 为 0xFC，公版 ota-code 为 0x05，两边需要修改为一致；ota 修改这个标志位在这个枚举里面：OTA_STA_ENTER

```
typedef enum
{
    OTA_STA_FAIL = 0x00,      // OTA 失败
    OTA_STA_SUC = 0x01,      // OTA 成功
    OTA_STA_WAITING = 0x02,  // 等待 OTA
    OTA_STA_CI = 0x03,       // OTA 语音端
    OTA_STA_DEV = 0x04,      // OTA 下挂设备中
    OTA_STA_ENTER = 0x05,    // 进入 OTA
} ota_state_t;
```

2. 将公版 sdk 里面的 write_ota_mcu_status 函数内容修为此个:

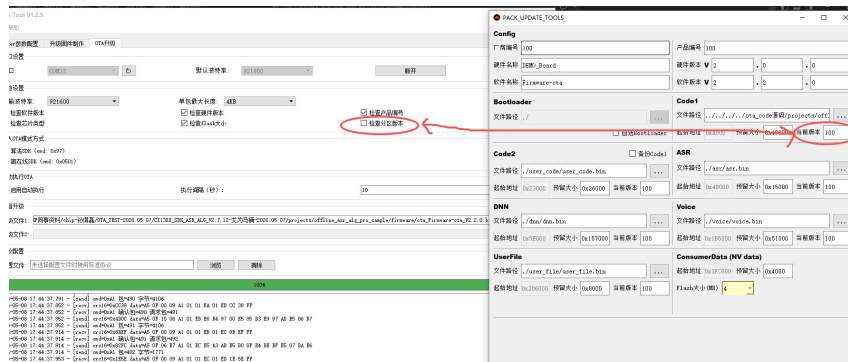
```
cinv_item_ret_t write_ota_mcu_status(uint8_t status)
{
    uint8_t flag = status;
    return cinv_item_write(NVDATA_ID_OTA_MCU_STATUS, sizeof(uint8_t), &flag);
}

cinv_item_ret_t write_ota_mcu_status(uint8_t status)
```

```
/*cinv_item_ret_t write_ota_mcu_status(uint8_t status)
{
    ota_nv_status_t ota_nv_status; 这个屏蔽掉
    ota_nv_status.status = status;
    ota_nv_status.ota_chip_type_1306 = OTA_CHIP_TYPE;
    ota_nv_status.timeout = OTA_TIMEOUT;
    ota_nv_status.retry_time = OTA_RETRY_TIME;
    ota_nv_status.log_uart_number = CONFIG_CI_LOG_UART;
    ota_nv_status.ota_uart_number = UART_PROTOCOL_NUMBER;
    ota_nv_status.ota_uart_baud = UART_PROTOCOL_BAUDRATE;
    return cinv_item_write(NVDATA_ID_OTA_MCU_STATUS, sizeof(ota_nv_status_t), &ota_nv_status);
}*/
cinv_item_ret_t write_ota_mcu_status(uint8_t status)
{
    uint8_t flag = status; 修改为这个
    return cinv_item_write(NVDATA_ID_OTA_MCU_STATUS, sizeof(uint8_t), &flag);
}

#endif // (UART_PROTOCOL_VER == 2)
```

OTA 测试工具检查分区版本 ✓ 上需要再打包的时候，分区版本号需要修改



客户需要在语音端进入到 ota 模式，为了兼容语音协议，可以直接在语音协议里面增加一条协议后将这段代码执行即可；

```
mprintf("Enter ota mode\r\n");
vTaskDelay(pdMS_TO_TICKS(15)); // 加点延迟 否则打印不完整
write_ota_mcu_status(OTA_STA_UPDATE);
dpmu_software_reset_system_config();
```

8.1.1 语音端进入 ota 后开始复位

语音端进入到 ota-code 后首先判断
ota 进入与否的标志位
OTA_STA_UPDATE，一致就进入到 OTA
模式；



1. 在进入到 ota-code 一切正常的情况下：ota 初始化完就会立刻发送协议到主端，告知主端已进入 ota 模式；发送函数：

```
// 初始化响应，上报 OTA 状态到主端
init_ota_ack_pack(&g_ota_ack_pack);
static void init_ota_ack_pack(cias_ota_pack_t *ota_pack)
{
    ota_pack->head0 = MSG_OTA_HEAD0;
    ota_pack->head1 = MSG_OTA_HEAD1;
    ota_pack->len0 = 0;
    ota_pack->len1 = OTA_ACK_STATUS_LEN; // 帧类型+校验值+帧尾+数据
    ota_pack->msg_type = MSG_TYPE_OTA_STATUS;
    ota_pack->tail = MSG_OTA_TAIL;
    ota_pack->data = pvPortMalloc(OTA_ACK_PACK_LEN);
    if (ota_pack->data == NULL)
```

```

{
    CI_ASSERT(0, "ota ack buffer malloc erro\n");
}
ota_pack->data[0] = g_ota_status.ota_state;
ota_pack->data[1] = g_ota_status.ota_reason;
get_ota_send_pack_crc(ota_pack);
}

```

客户需要修改协议告知主端进入到 ota 就在这儿修改为自己的协议即可：

公版 ota-code 发送 A5 0F 00 06 A2 02 01 FB 8D FF 到主端告知语音端已进入到 ota;

8.1.2 语音端开始 ota

主端收到语音端进入到
ota 模式



1. 主端发送: A5 0F 00 11 A0 01 01 01 01 01 00 02 02 00 00 0E 10 00 24 C7 FF

`MSG_TYPE_OTA_VER_PARM_SET = 0xA0,` // 版本及参数命令设置 host-》ci

主端发送版本和参数设置命令 版本检验与否 软件版本号 2.2.0 000e1000 波特率是 921600bps;

```

case MSG_TYPE_OTA_VER_PARM_SET:
{
    // mprintf("OTA MSG_TYPE_OTA_VER_PARM_SET\r\n");
    uint8_t ota_version_local[4];
    result = parse_and_check_ota_ver_param(ota_recv_pack.data, ota_version_local);
    get_ota_ver_param_ack_pack(&g_ota_ack_pack, result, ota_version_local);
    vmup_send_ota_packet_to_host(&g_ota_ack_pack);
    break;
}

```

`parse_and_check_ota_ver_param`: 解析保存主端参数

客户修改这儿的协议后在此处对应语音接收协议

2. 语音回复主端:

`get_ota_ver_param_ack_pack(&g_ota_ack_pack, result, ota_version_local);`

公版发送的协议为: A5 0F 00 09 A0 01 02 00 09 00 B0 58 FF 客户修改就在这个函数修改:

/*****/

3. 主端发送 A5 0F 00 05 A3 00 64 0D FF 开始 ota 数据传输

OTA 语音端: 0x00

OTA 下挂设备: 0x01

4. 语音端回复: A5 0F 00 06 A2 03 00 D8 9D FF 03: 升级语音芯片中

0x00: 升级失败; 0x01: 升级成功; 0x02: 等待升级; 0x03: 升级语音中; 0x04: 升级下挂设备中+原因 (1 字节)

/*****/

5. 主端开始发送 : A5 0F 10 06 A1 00 00 00 20 00 05 75 73 65 72 5F 63 6F 64 65 32 61 73 72 64 6E 6E 76 6F 69 63 65 75 73 65 72 5F 66 69 6C 65 3D 0D

6. 语音端回复 A5 0F 00 09 A1 01 00 00 00 00 A2 08 FF

/*****/

7. 主端继续发送完毕后, 语音端回复: A5 0F 00 06 A2 01 00 BE FF FF 01: 表示升级成功 芯片开始自动重启

客户主端和语音端修改协议后在 ota-code 里面修改对应协议 语音端修改在 ota-code

```
void ota_main_task(void *pvParameter)
```